

Banque MPI Par Points d'Evaluation - Sections 10 / 按考察点分层的 MPI 题库 - 第 10 节

10. Grands Problemes a Questions Enchainees / 10. 连环追问算法大题

Chaque grand probleme est redige dans un style proche des sujets des annees precedentes : contexte, taches a effectuer, pseudo-code et analyse de complexite. / 每个大题都按往年试卷风格书写：先给背景，再给任务、伪代码和复杂度分析。

Pour eviter les redites, cette section conserve seulement des grands problemes representatifs ; les enonces trop proches d'un meme schema ont ete supprimes. / 为避免重复，本节只保留具有代表性的算法大题；过于接近同一通信模板的题目已删除。

Grand Probleme 1 : Schema unifie de reduction globale / 大题 1：全局归约统一模型题

On considere un tableau de taille N , initialise sur le processus de rang 0 puis distribue sur P processus MPI. Le but de ce probleme est de construire un schema general de resolution valable pour plusieurs variantes classiques : somme globale, maximum global, minimum global, moyenne globale, produit scalaire, comptage de valeurs positives. / 现考虑一个大小为 N 的数组，先在 rank 0 上初始化，再分发给 P 个 MPI 进程。本题目标是建立一个通用解题模型，可用于多种经典变体：全局求和、全局最大值、全局最小值、全局平均值、点积、正数个数统计。

10.01 [Algorithmme:Reduction globale] Decrivez le patron general de l'algorithme parallele : distribution, calcul local, communication finale et affichage. / 描述该并行算法的通用模式：分发、局部计算、最终通信和输出。

Reponse : Une reponse algorithmique complete peut etre structuree ainsi : 1. initialiser MPI et recuperer `rang` et `P` ; 2. sur la racine, preparer les donnees globales ; 3. diffuser les parametres globaux puis distribuer les blocs de donnees ; 4. effectuer le calcul local sur chaque processus ; 5. combiner les resultats locaux par une communication finale de type `Reduce` ou `Allreduce` ; 6. faire afficher le resultat par la racine, puis terminer MPI. / 答：完整算法可写成：1. 初始化 MPI 并获得 `rang` 和 `P` ; 2. 根进程准备全局数据；3. 广播参数并分发数据块；4. 各进程执行局部计算；5. 通过 `Reduce` 或 `Allreduce` 合并局部结果；6. 根进程输出并结束 MPI。

10.02 [Communication:Reduction globale] Indiquez quelles primitives collectives conviennent dans le cas general et expliquez quand utiliser `Reduce`, `Allreduce`, `Scatter`, `Gather` ou `Bcast`. / 指出一般情况下应使用哪些集合通信，并说明何时用 `Reduce`、`Allreduce`、`Scatter`、`Gather` 或 `Bcast`。

Reponse : `Bcast` sert a diffuser un parametre global comme `N`. `Scatter` sert a distribuer des blocs de donnees. `Reduce` sert a combiner les resultats locaux sur la racine. `Allreduce` est prefere si tous doivent connaitre le resultat. `Gather` sert a recolter explicitement des blocs ou des resultats individuels. / 答：`Bcast` 用于广播全局参数，`Scatter` 用于分发数据块，`Reduce` 用于在根进程上合并局部结果；若所有进程都要结果，则用 `Allreduce`；若要显式回收各局部块，可用 `Gather`。

10.03 [Impl:Reduction globale] Ecrivez un pseudo-code MPI generique correspondant a ce patron. / 写出与这一模式对应的通用 MPI 伪代码。

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int rang, P, N = 8, local_n, i; double *tab = NULL, *bloc, res_loc = 0.0, res_glob = 0.0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    local_n = N / P; bloc = (double *)malloc(local_n * sizeof(double));
    if (rang == 0) {
        tab = (double *)malloc(N * sizeof(double));
        for (i = 0; i < N; i++) tab[i] = i + 1;
    }
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter(tab, local_n, MPI_DOUBLE, bloc, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    for (i = 0; i < local_n; i++) res_loc += bloc[i];
    MPI_Reduce(&res_loc, &res_glob, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rang == 0) { printf("resultat global = %.1f\n", res_glob); free(tab); }
    free(bloc); MPI_Finalize(); return 0;
}
```

10.04 [Complexite:Reduction globale] Donnez la complexite locale du calcul et la complexite de communication de ce schema. / 给出该模式的局部计算复杂度和通信复杂度。

Reponse : Si les N donnees sont bien reparties, le calcul local est en $O(N / P)$. La communication finale de type reduction est en $O(\log P)$ dans une estimation simple. / 答：若 N 个数块均匀分配，则局部计算复杂度是 $O(N / P)$ ；最终归约通信简单估计为 $O(\log P)$ 。

10.05 [Variation:Reduction globale] Expliquez comment adapter ce meme schema si l'on remplace la somme par un maximum, un minimum, une moyenne, un produit scalaire ou un comptage de valeurs positives. / 说明如果把“求和”改成“最大值、最小值、平均值、点积、统计正数个数”，应如何复用这一模式。

Reponse : On garde la meme structure generale, mais on change le calcul local et l'operateur de reduction. Par exemple, on utilise `MPI_MAX` pour un maximum, `MPI_MIN` pour un minimum, on additionne localement pour une moyenne puis on divise a la fin, on calcule localement les produits `x[i] * y[i]` pour un produit scalaire, et on compte localement les positifs pour un comptage. / 答：总体框架不变，只需改本地计算内容和归约算子：最大值用 `MPI_MAX`，最小值用 `MPI_MIN`，平均值先求总和再除以总数，点积先本地算乘积和，统计正数则本地计数后归约。

10.06 [Variation:Reduction globale] Si tous les processus doivent connaitre le resultat final, quelle modification faites-vous au schema precedent ? / 如果所有进程都必须知道最终结果，应如何修改该模式？

Reponse : On remplace `MPI_Reduce` par `MPI_Allreduce`, ou bien on ajoute un `MPI_Bcast` apres `MPI_Reduce`. / 答：可以直接把 `MPI_Reduce` 换成 `MPI_Allreduce`，或者在 `MPI_Reduce` 后再做一次 `MPI_Bcast`。

10.07 [Repartition:Reduction globale] Supposons maintenant que N ne soit pas divisible par P . Proposez deux strategies correctes pour repartir les donnees et expliquez dans quel cas `MPI_Scatterv` / `MPI_Gatherv` deviennent utiles. / 现在假设 N 不能被 P 整除。请提出两种正确的数据分配策略，并说明在什么情况下 `MPI_Scatterv` / `MPI_Gatherv` 会变得有用。

Reponse : Une premiere strategie consiste a donner $N / P + 1$ elements a certains processus et N / P aux autres. Une deuxieme consiste a laisser la racine traiter le reste. Des que les tailles locales ne sont plus toutes identiques, `MPI_Scatterv` et `MPI_Gatherv` deviennent utiles. / 答：一种方法是让部分进程多拿 1 个元素，其余进程拿 N / P 个；另一种方法是让根进程额外处理余数部分。只要块大小不一致，就需要 `MPI_Scatterv` / `MPI_Gatherv`。

Grand Probleme 2 : Produit scalaire avec lecture sur le rang 0 / 大题 2：由 rank 0 读入的点积

On impose que seules les fonctions de lecture soient appelees sur le processus de rang 0, puis que les donnees soient distribuees aux autres processus. / 题目要求：读入函数只能在 rank 0 上调用，之后再把数据分发给其他进程。

10.13 [Justification:Produit scalaire] Expliquez pourquoi cette contrainte est frequente en MPI. / 解释为什么这种约束在 MPI 题目中很常见。

Reponse : Parce qu'en pratique la lecture de fichier ou l'initialisation centralisee est souvent plus simple a gerer sur une seule tache. Ensuite, MPI sert a distribuer les donnees. / 答：因为实际中让单个进程负责读文件或初始化通常更简单，然后再用 MPI 把数据分发出去。

10.14 [Communication:Produit scalaire] Indiquez quelles donnees doivent etre diffusees a tous et lesquelles doivent etre distribuees par blocs. / 指出哪些数据应广播给所有进程，哪些应按块分发。

Reponse : Une valeur globale comme `N` doit etre diffusee avec `MPI_Bcast`. En revanche, les tableaux `x` et `y` doivent etre distribues par blocs avec `MPI_Scatter`. / 答：像 `N` 这样的全局参数应使用 `MPI_Bcast` 广播；而向量 `x`、`y` 则应按块用 `MPI_Scatter` 分发。

10.15 [Algorithmme:Produit scalaire] Ecrivez les grandes etapes du programme MPI. / 写出该 MPI 程序的主要步骤。

Reponse : Les grandes etapes sont : 1. initialiser MPI ; 2. faire lire `N`, `x` et `y` par le rang 0 ; 3. diffuser `N` a tous les processus ; 4. distribuer les morceaux de `x` et `y` avec `MPI_Scatter` ou `MPI_Scatterv` ; 5. calculer le produit scalaire local ; 6. sommer tous les resultats locaux avec `MPI_Reduce` ; 7. afficher le resultat final sur la racine. / 答：主要步骤是：1. 初始化 MPI；2. 由 rank 0 读入 `N`、`x`、`y`；3. 广播 `N`；4. 分发 `x` 和 `y` 的局部块；5. 计算本地点积；6. 用 `MPI_Reduce` 合并；7. 根进程输出结果。

10.16 [Impl:Produit scalaire] Ecrivez un pseudo-code de `main`. / 写出 `main` 的伪代码。

Reponse :

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int rang, P, N = 8, local_n, i; double *x = NULL, *y = NULL, *local_x, *local_y, res_loc = 0.0, res_glob = 0.0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    local_n = N / P; local_x = (double *)malloc(local_n * sizeof(double)); local_y = (double *)malloc(local_n * sizeof(double));
    if (rang == 0) {
        x = (double *)malloc(N * sizeof(double));
        y = (double *)malloc(N * sizeof(double));
        for (i = 0; i < N; i++) { x[i] = i + 1; y[i] = 1.0; }
    }
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter(x, local_n, MPI_DOUBLE, local_x, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Scatter(y, local_n, MPI_DOUBLE, local_y, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    for (i = 0; i < local_n; i++) res_loc += local_x[i] * local_y[i];
    MPI_Reduce(&res_loc, &res_glob, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rang == 0) { printf("produit scalaire = %.1f\n", res_glob); free(x); free(y); }
    free(local_x); free(local_y); MPI_Finalize(); return 0;
}

```

10.17 [Complexite:Produit scalaire] Donnez la complexite de l'application. / 给出该应用复杂度。

Reponse : Le calcul local est en $O(N / P)$, et la reduction finale est en $O(\log P)$ dans une estimation simple. / 答：局部计算复杂度是 $O(N / P)$ ，最终归约简单估计为 $O(\log P)$ 。

Grand Probleme 3 : Normalisation par somme globale / 大题 3：按全局和归一化

On souhaite normaliser un vecteur distribue en divisant chaque element local par la somme globale de tous les elements. / 现在希望对一个分布式向量做归一化：每个局部元素都除以全局总和。

10.18 [Algorithme:Normalisation globale] Decrivez le calcul local initial necessaire. / 描述所需的初始局部计算。

Reponse : Le calcul local initial suit l'idee suivante : 1. chaque processus parcourt son bloc local ; 2. il additionne ses elements dans une variable de somme locale ; 3. cette somme locale est ensuite prete a etre combinee globalement. / 答：初始局部计算思路是：1. 每个进程遍历自己的局部数据块；2. 把元素累加到局部和变量中；3. 再把这个局部和用于后续全局合并。

10.19 [Communication:Normalisation globale] Comment obtenir la somme globale des elements ? / 如何得到所有元素的全局和？

Reponse : On utilise une reduction, par exemple `MPI_Reduce` ou directement `MPI_Allreduce`. / 答：可以通过归约得到全局总和，例如 `MPI_Reduce` 或直接用 `MPI_Allreduce`。

10.20 [Communication:Normalisation globale] Comment rendre cette somme disponible sur tous les processus ? / 如何让所有进程都得到这个总和？

Reponse : Soit on fait `MPI_Reduce` puis `MPI_Bcast`, soit on utilise directement `MPI_Allreduce`. / 答：可以先 `MPI_Reduce` 再 `MPI_Bcast`，也可以直接使用 `MPI_Allreduce`。

10.21 [Algorithme:Normalisation globale] Decrivez la phase de normalisation locale. / 描述本地归一化阶段。

Reponse : La normalisation locale se fait en trois temps : 1. attendre que la somme globale soit disponible localement ; 2. parcourir a nouveau le bloc local ; 3. remplacer chaque valeur locale `x[i]` par `x[i] / somme_globale`. / 答：局部归一化分三步：1. 等待每个进程都得到全局和；2. 再次遍历本地块；3. 将每个局部元素替换为 `x[i] / somme_globale`。

10.22 [Complexite:Normalisation globale] Donnez la complexite globale. / 给出整体复杂度。

Reponse : Le calcul local initial et la normalisation locale sont en $O(N / P)$, et la communication globale est en $O(\log P)$ dans une estimation simple. / 答：初始局部求和和归一化都是 $O(N / P)$ ，全局通信简单估计为 $O(\log P)$ 。

10.23 [Impl:Normalisation globale] Ecrivez un pseudo-code MPI de la normalisation distribuee. / 写出该分布式归一化的 MPI 伪代码。

Reponse :

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int rang, P, N = 8, local_n, i;
    double *x = NULL, *local_x, somme_loc = 0.0, somme_glob = 0.0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
    local_n = N / P;
    local_x = (double *)malloc(local_n * sizeof(double));
    if (rang == 0) {
        x = (double *)malloc(N * sizeof(double));
        for (i = 0; i < N; i++) x[i] = i + 1;
    }
    MPI_Scatter(x, local_n, MPI_DOUBLE, local_x, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    for (i = 0; i < local_n; i++) somme_loc += local_x[i];
    MPI_Allreduce(&somme_loc, &somme_glob, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
    for (i = 0; i < local_n; i++) local_x[i] = local_x[i] / somme_glob;
    if (rang == 0) free(x);
    free(local_x);
    MPI_Finalize();
    return 0;
}

```

Grand Probleme 4 : Echange en anneau sur P etapes / 大题 4：P 步环形传递

On veut realiser un echange en anneau : a chaque etape, chaque processus envoie une valeur a son voisin de droite et recoit une valeur de son voisin de gauche. L'operation est repetee `P` fois. / 现在要实现一个环形交换：每一步，每个进程向右邻居发送一个值，并从左邻居接收一个值。该操作共重复 `P` 次。

10.28 [Algorithme:Anneau] Definissez precisement les voisins gauche et droit. / 精确定义左邻居和右邻居。

Reponse : La definition algorithmique est : 1. recuperer `rang` et `P` ; 2. definir `gauche = (rang + P - 1) % P` pour assurer le retour cyclique du rang `0` vers `P - 1` ; 3. definir `droite = (rang + 1) % P` pour assurer le retour cyclique du rang `P - 1` vers `0`. / 答：算法定义为：1. 获取 `rang` 和 `P` ; 2. 令 `gauche = (rang + P - 1) % P` ; 3. 令 `droite = (rang + 1) % P`，从而形成闭环。

10.29 [Algorithme:Anneau] Proposez une version correcte de l'algorithme, bloquante ou non bloquante. / 提出一个正确的阻塞版或非阻塞版算法。

Reponse : Une version correcte suit ces etapes : 1. calculer `gauche` et `droite` ; 2. initialiser la valeur courante ; 3. a chaque etape, poster `MPI_Isend` vers `droite` et `MPI_Irecv` depuis `gauche` ; 4. attendre les deux communications avec `MPI_Wait` ; 5. remplacer la valeur courante par celle recue ; 6. repeter jusqu'a la fin de

l'anneau. / 答：正确算法是：1. 计算左右邻居；2. 初始化当前值；3. 每一步向右 `MPI_Isend`、从左 `MPI_Irecv`；4. 用 `MPI_Wait` 等待完成；5. 用收到的新值更新；6. 重复直到结束。

10.30 [Analyse:Anneau] Expliquez ce qui circule dans l'anneau a chaque etape. / 解释每一步在环中传递的内容。

Reponse : A chaque etape, chaque processus transmet sa valeur courante a droite et recoit une nouvelle valeur venant de gauche, qui devient sa valeur courante pour l'etape suivante. / 答：每一步中，每个进程把当前值传给右邻居，并从左邻居接收一个新值，作为下一步的当前值。

10.31 [Impl:Anneau] Ecrivez le pseudo-code MPI. / 写出 MPI 伪代码。

Reponse :

```
/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, P, gauche, droite, val, tmp, etape; MPI_Request req1, req2;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_size(MPI_COMM_WORLD, &P);
    gauche = (rang + P - 1) % P; droite = (rang + 1) % P; val = rang;
    for (etape = 0; etape < P; etape++) {
        MPI_Isend(&val, 1, MPI_INT, droite, 0, MPI_COMM_WORLD, &req1);
        MPI_Irecv(&tmp, 1, MPI_INT, gauche, 0, MPI_COMM_WORLD, &req2);
        MPI_Wait(&req1, MPI_STATUS_IGNORE); MPI_Wait(&req2, MPI_STATUS_IGNORE); val = tmp;
    }
    printf("rang %d finit avec %d\n", rang, val);
    MPI_Finalize(); return 0;
}
```

10.32 [Complexite:Anneau] Donnez la complexite en temps et en nombre de messages. / 给出时间复杂度和消息数复杂度。

Reponse : Il y a `P` etapes, et a chaque etape on echange `P` messages au total. Le nombre total de messages est donc $O(P^2)$ sur l'ensemble des etapes. / 答：总共做 `P` 步，每步全局有 `P` 条消息交换，所以整个过程总消息数是 $O(P^2)$ 。

Grand Probleme 5 : Statistique differente dans groupes pair/impair / 大题 5：奇偶组做不同统计

On souhaite separer les processus pairs et impairs dans deux sous-communicateurs distincts. Les processus pairs calculent une somme de groupe, tandis que les processus impairs calculent un maximum de groupe. / 现在希望把奇数秩和偶数秩分到两个不同的子 communicator 中。偶数组计算组内总和，奇数组计算组内最大值。

10.33 [Communication:Groupes pair/impair] Expliquez comment construire les deux communicateurs avec `MPI_Comm_split`. / 解释如何用 `MPI_Comm_split` 构建两个 communicator。

Reponse : Il suffit d'utiliser `rang % 2` comme color et `rang` comme key . Les pairs iront dans un groupe, les impairs dans l'autre. / 答：可直接把 `rang % 2` 作为 color , `rang` 作为 key ；偶数进程进入一个组，奇数进程进入另一个组。

10.34 [Algorithme:Groupes pair/impair] Precisez le calcul effectue dans chacun des groupes. / 说明每个组中执行的计算。

Reponse : L'algorithme est le suivant : 1. separer les processus avec `MPI_Comm_split` ; 2. dans le groupe pair, calculer un resultat local puis lancer une reduction `MPI_SUM` ; 3. dans le groupe impair, calculer un resultat local puis lancer une reduction `MPI_MAX` ; 4. exploiter ou afficher le resultat sur la racine locale de chaque sous-groupe. / 答：算法步骤是：1. 用 `MPI_Comm_split` 分组；2. 偶数组内先计算局部值再做 `MPI_SUM` ；3. 奇数组内先计算局部值再做 `MPI_MAX` ；4. 在各子组本地根进程使用或输出结果。

10.35 [Impl:Groupes pair/impair] Ecrivez le pseudo-code de la solution. / 写出该方案的伪代码。

Reponse :

```
/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, local_rank, val, res; MPI_Comm comm_pi;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Comm_split(MPI_COMM_WORLD, rang % 2, rang, &comm_pi);
    MPI_Comm_rank(comm_pi, &local_rank);
    val = rang;
    if (rang % 2 == 0) MPI_Reduce(&val, &res, 1, MPI_INT, MPI_SUM, 0, comm_pi);
    else MPI_Reduce(&val, &res, 1, MPI_INT, MPI_MAX, 0, comm_pi);
    if (local_rank == 0) printf("groupe %d -> %d\n", rang % 2, res);
    MPI_Comm_free(&comm_pi); MPI_Finalize(); return 0;
}
```

10.36 [Justification:Groupes pair/impair] Comment les resultats de groupe peuvent-ils ensuite etre recuperes ou affiches au niveau global ? / 这些组内结果之后如何在全局层面回收或输出？

Reponse : Chaque racine locale peut afficher son resultat dans son sous-communicateur, ou bien l'envoyer ensuite au rang `0` de `MPI_COMM_WORLD` . / 答：每个子组的本地 root 可以直接输出结果，也可以再把结果发回 `MPI_COMM_WORLD` 的 rank 0 做统一汇总。

10.37 [Complexite:Groupes pair/impair] Donnez une analyse simple de la complexite. / 给出一个简单的复杂度分析。

Reponse : Chaque reduction se fait sur environ $P / 2$ processus. Dans une estimation simple, le cout de communication reste logarithmique en la taille du groupe. / 答：每个归约大约只在 $P / 2$ 个进程上进行，简单估计下其通信复杂度仍是组大小上的对数级。

Grand Probleme 6 : Diffusion dans sous-groupes / 大题 6：在子组内广播

Apres partition des processus en plusieurs groupes, on veut diffuser une valeur differente dans chaque sous-groupe. / 在完成进程分组后，要求在每个子组内部广播不同的值。

10.38 [Algorithme:Diffusion en sous-groupes] Decrivez le principe du regroupement. / 描述分组原理。

Reponse : Le principe est : 1. choisir une regle de partition, par exemple la parite ou `rang % k` ; 2. calculer la couleur associee a chaque processus ; 3. appeler `MPI_Comm_split` avec cette couleur ; 4. obtenir ainsi un sous-communicateur independant pour chaque groupe ; 5. effectuer ensuite les diffusions localement dans chaque sous-groupe. / 答：分组原理是：1. 先选定划分规则，如奇偶或 `rang % k` ；2. 为每个进程计算 color ；3. 调用 `MPI_Comm_split` ；4. 由此得到每个组自己的子 communicator ；5. 之后在各子组内单独广播。

10.39 [Communication:Diffusion en sous-groupes] Comment choisir la racine locale dans chaque sous-groupe ? / 如何在每个子组内选择本地根进程？

Reponse : On choisit un rang local dans le sous-communicateur, souvent `0` . / 答：应在子 communicator 内选一个本地 rank 作为 root，通常选择 `0`。

10.40 [Impl:Diffusion en sous-groupes] Ecrivez le pseudo-code de creation du communicator puis de l'appel a `MPI_Bcast` . / 写出 communicator 创建与 `MPI_Bcast` 调用的伪代码。

Reponse :

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, color, local_rank, x = -1; MPI_Comm sous_comm;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    color = rang % 2;
    MPI_Comm_split(MPI_COMM_WORLD, color, rang, &sous_comm);
    MPI_Comm_rank(sous_comm, &local_rank);
    if (local_rank == 0) x = 100 + color;
    MPI_Bcast(&x, 1, MPI_INT, 0, sous_comm);
    printf("rang %d dans groupe %d a reçu %d\n", rang, color, x);
    MPI_Comm_free(&sous_comm); MPI_Finalize(); return 0;
}

```

10.41 [Definition:Rang global vs local] Expliquez la difference entre un rang global et un rang local. / 解释全局 rank 与局部 rank 的区别。

Reponse : Le rang global est le rang du processus dans `MPI_COMM_WORLD`. Le rang local est son rang dans un sous-communicateur cree ensuite. / 答：全局 rank 是进程在 `MPI_COMM_WORLD` 中的编号；局部 rank 是它在某个后续创建的子 communicator 中的编号。

10.42 [Complexite:Diffusion en sous-groupes] Donnez une estimation qualitative du cout de communication. / 给出通信代价的定性估计。

Reponse : Le cout depend de la taille de chaque sous-groupe, pas de `P` tout entier. Plus les groupes sont petits, plus la diffusion locale est legere. / 答：通信开销取决于各子组的规模，而不是整个 `P`；子组越小，组内广播代价越低。

Grand Probleme 7 : Reception d'un message de taille inconnue / 大题 7：接收未知大小消息

Un processus doit recevoir un message, mais il ne connait pas a l'avance le nombre d'elements a recevoir. / 某进程需要接收一条消息，但事先不知道要接收多少个元素。

10.43 [Analyse:Taille inconnue] Expliquez en quoi cette situation pose un probleme pratique. / 解释这种情况为什么会带来实际问题。

Reponse : `MPI_Recv` a besoin d'un tampon deja alloue et d'une borne de taille. Si la vraie taille du message est inconnue, on risque soit de sous-dimensionner le tampon, soit de sur-allouer inutilement. / 答：`MPI_Recv` 需要事先分配接收缓冲区并给出大小上界；若真实消息长度未知，就可能缓冲区过小或无谓地过大。

10.44 [Algorithme:Taille inconnue] Proposez une strategie MPI correcte pour gerer ce cas, d'abord avec `MPI_Probe`, puis avec `MPI_Iprobe`. / 分别用 `MPI_Probe` 和 `MPI_Iprobe` 给出正确的 MPI 策略。

Reponse : La strategie correcte est : 1. sonder le message avec `MPI_Probe` ou, en version non bloquante, boucler avec `MPI_Iprobe` ; 2. recuperer la taille exacte avec `MPI_Get_count` ; 3. allouer un tampon de taille adaptee ; 4. recevoir effectivement les donnees avec `MPI_Recv` ; 5. traiter puis liberer le tampon si necessaire. / 答：正确策略是：1. 用 `MPI_Probe` 或循环 `MPI_Iprobe` 探测消息；2. 用 `MPI_Get_count` 得到真实长度；3. 分配合适大小的缓冲区；4. 再用 `MPI_Recv` 真正接收；5. 最后处理并按需释放缓冲区。

10.45 [Impl:Taille inconnue] Ecrivez le schema standard `Probe/Iprobe -> Get_count -> allocation -> Recv`. / 写出标准流程： `Probe/Iprobe -> Get_count -> 分配内存 -> Recv`。

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int rang, n, *buf; MPI_Status sta;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    if (rang == 0) {
        int msg[3] = {10, 20, 30};
        MPI_Send(msg, 3, MPI_INT, 1, 7, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &sta);
        MPI_Get_count(&sta, MPI_INT, &n);
        buf = (int *)malloc(n * sizeof(int));
        MPI_Recv(buf, n, MPI_INT, sta.MPI_SOURCE, sta.MPI_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("rang 1 a reçu %d entiers\n", n); free(buf);
    }
    MPI_Finalize(); return 0;
}

```

10.46 [Definition:Get_count] Quelles fonctions MPI sont utiles ici, et quel est le role exact de `MPI_Get_count` ? / 这里会用到哪些 MPI 函数？ `MPI_Get_count` 的确切作用是什么？

Reponse : Les fonctions utiles sont `MPI_Probe` ou `MPI_Iprobe`, puis `MPI_Get_count`, puis `MPI_Recv`. `MPI_Get_count` sert a determiner combien d'elements d'un type MPI donne ont reellement ete recus ou annonces. / 答：这里主要用到 `MPI_Probe` / `MPI_Iprobe`、`MPI_Get_count` 和 `MPI_Recv`；其中 `MPI_Get_count` 的作用是得到某种 MPI 类型实际对应的元素个数。

10.47 [Analyse:Taille inconnue] Expliquez les erreurs classiques a eviter si l'on essaye de recevoir directement sans connaitre la vraie taille du message. / 解释如果在不知道真实消息大小时直接接收，会有哪些典型错误。

Reponse : On peut recevoir dans un tampon trop petit, provoquer une erreur de troncature, ou lire un contenu mal interprete. / 答：常见错误包括缓冲区过小、截断错误，以及按错误大小理解消息内容。

Grand Probleme 8 : Envoi d'une colonne de matrice / 大题 8：发送矩阵的一列

Une matrice est stockee en memoire ligne par ligne sur un processus. On veut envoyer une colonne complete de cette matrice a un autre processus sans recopier manuellement les donnees dans un tampon temporaire. / 一个矩阵按行存储在某进程内存中。现在希望把该矩阵的一整列发送给另一个进程，并避免手工把数据复制到临时缓冲区中。

10.48 [Analyse:Colonne de matrice] Expliquez pourquoi les elements de la colonne ne sont pas contigus en memoire. / 解释为什么这一列元素在内存中不是连续的。

Reponse : Parce que la matrice est stockee ligne par ligne. Deux elements consecutifs d'une meme colonne sont separees en memoire par toute la largeur de la ligne. / 答：因为矩阵按行存储，同一列的相邻两个元素在内存中会隔着整行的跨度，所以不连续。

10.49 [Communication:Colonne de matrice] Quel type derive MPI faut-il utiliser ? / 应使用哪种 MPI 派生类型？

Reponse : En pratique on utilise `MPI_Type_vector`. / 答：通常应使用 `MPI_Type_vector`。

10.50 [Impl:Colonne de matrice] Ecrivez les etapes de creation et de validation du type derive. / 写出创建并提交该派生类型的步骤。

Reponse :

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Datatype colonne; int rang; int nb_lignes = 4, nb_colonnes = 5;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    MPI_Type_vector(nb_lignes, 1, nb_colonnes, MPI_DOUBLE, &colonne);
    MPI_Type_commit(&colonne);
    if (rang == 0) {
        printf("type derive colonne cree et valide\n");
    }
    MPI_Type_free(&colonne);
    MPI_Finalize();
    return 0;
}

```

10.51 [Impl:Colonne de matrice] Montrez comment utiliser ce type dans une communication MPI. / 说明如何在 MPI 通信中使用该类型。

Reponse :

```

/* Exemple MPI minimal. */
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int rang, i, nb_lignes = 4, nb_colonnes = 5, j = 2; double A[4][5], buf[4]; MPI_Datatype colonne;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
    for (i = 0; i < nb_lignes; i++) {
        for (int k = 0; k < nb_colonnes; k++) {
            A[i][k] = 10.0 * i + k;
        }
    }
    MPI_Type_vector(nb_lignes, 1, nb_colonnes, MPI_DOUBLE, &colonne);
    MPI_Type_commit(&colonne);
    if (rang == 0) {
        MPI_Send(&A[0][j], 1, colonne, 1, 0, MPI_COMM_WORLD);
    } else if (rang == 1) {
        MPI_Recv(buf, nb_lignes, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        for (i = 0; i < nb_lignes; i++) {
            printf("buf[%d] = %.1f\n", i, buf[i]);
        }
    }
    MPI_Type_free(&colonne);
    MPI_Finalize();
    return 0;
}

```

10.52 [Justification:Colonne de matrice] Expliquez pourquoi il faut ensuite liberer le type. / 解释为什么随后需要释放该类型。

Reponse : Comme pour tout objet MPI derive, il faut liberer le type pour relacher correctement la ressource MPI associee. / 答：和其他 MPI 派生对象一样，使用完后应释放该类型，以正确回收 MPI 资源。